KURUKSHETRA KART MASTER BUILD DOCUMENT

IC Go-Kart Telemetry, Digital Twin & Predictive Systems

Prepared by: Kurukshetra Racing, SKCET
Competition: EKVC Season 4 — Innovation Round
Team: Embedded Systems, Telemetry, Data Analytics, Dynamics & Testing
Document Version: 1.0 Master Build Manual
Timeline: Semester-based (3–6 months, phase-gated)

EXECUTIVE SUMMARY

Kurukshetra Kart is a complete telemetry-to-prediction platform built around a low-cost embedded stack (ESP32 only) that transforms real go-kart performance data into a validated physics-based digital twin. The system closes an engineering feedback loop: measure real kart behavior → build a model → test virtual scenarios → validate predictions → improve real setup.

Core Innovation: The closed-loop integration of predictive collision awareness (time-to-contact driven) with a calibrated physics simulator, supported by a disciplined data pipeline and honest validation methodology. This is not a sensor-by-sensor gadget—it is an end-to-end engineering tool.

Architecture in one line:
ESP32 collects nine core telemetry streams → logs locally to SD card → sends live subset via wireless to pit → Python ingests and models → simulator tests alternate conditions → results feed back to setup optimization.

Key Technical Commitments:

- **Controller:** ESP32 (dual-core, 240 MHz) — drop Arduino Nano entirely
- **Data Strategy:** Full local logging (SD card), compact wireless subset (10–20 Hz)
- **Simulation:** Physics-based point-mass model, validated against real telemetry

- **Track Position:** Flagged as open R&D problem; three candidate solutions proposed (RTK-GPS, dead reckoning, sensor fusion)
- **Cost:** Realistic BOM ₹18,000–24,000 (core hardware + wireless + data + integration)
- **Delivery:** Layered: V1 (core telemetry + display) → V2 (wireless + dashboard) → V3 (digital twin + advanced ML)

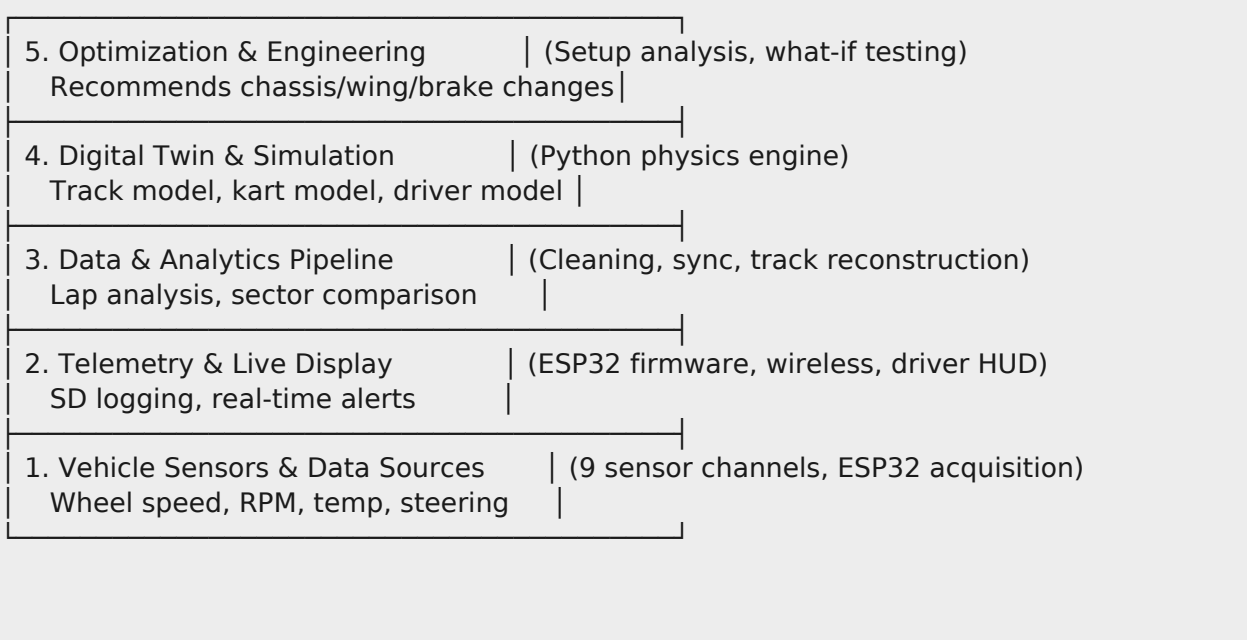
Team Structure: Seven functional subteams with clear ownership; no overlapping authority on subsystems.

---

1. SYSTEM ARCHITECTURE OVERVIEW

1.1 The Five-Layer Stack

Data flows upward through five layers. Each layer has a distinct owner team and can operate independently.



Critical Design Principle:
The kart continues to log to SD even if wireless fails. Wireless telemetry is useful; SD logging is mandatory.

1.2 Data Flow Summary

- **On-kart:** Sensors → ESP32 → simultaneous SD logging + wireless packet creation

- **To pit:** Compact live packet (speed, RPM, temperature, steering, G-force, TTC, warning flag) at 10–20 Hz
- **Post-run:** Full SD dataset (all channels, 50–100 Hz) imported into Python
- **Analysis:** Cleaned dataset → track reconstruction → lap/sector alignment → virtual modeling
- **Simulation:** Model tested against real data; predictions compared to measured corner speeds
- **Feedback:** Validated setup recommendations return to real kart for next run

---

2. HARDWARE MASTER PLAN

2.1 Core Components — V1 (Semester Target)

System	Component	Purpose	Qty	Est. Cost (₹)	Notes
Controller	ESP32 Dev Board	Central acquisition, filtering, comms	1	400–600	Dual-core, 240 MHz; drop Arduino Nano
Wheel Speed	Hall-effect sensor + magnet	Vehicle speed, wheel slip	1–2	80–150 each	Proven on multiple karts
Engine RPM	Inductive pickup / tach sensor	Engine operating point	1	150–300	Signal conditioning required
Engine Temp	DS18B20 probe	Thermal monitoring	1	80–120	±0.5°C precision
Steering Angle	Rotary potentiometer / encoder	Driver input, corner analysis	1	50–100	Linear pot on steering column

System	Component	Purpose	Qty	Est. Cost (₹)	Notes
G-Force / IMU	MPU6050 or ICM-42688	Accel, gyro, yaw rate	1	120–200	Essential for dynamics validation
Obstruction / TTC	HC-SR04 (V1) or ToF (V2)	Closing speed, collision warning	1	80–150	Ultrasonic for proof-of-concept
Throttle Position	Hall throttle sensor or pot	Driver input for modeling	1	100–200	Intake manifold mount
Brake Pressure	Brake line transducer	Braking dynamics (V2+)	1	400–800	V2 addition; high reliability needed
Data Logging	MicroSD module + card	Full dataset archival	1	150–250	32GB card, sustained 50 Hz logging
Display (driver)	0.96" OLED (I2C)	Steering-wheel HUD	1	180–250	Minimal; driver sees speed/RPM/alert only
Wireless Link	nRF24L01+ or ESP-NOW	Live telemetry to pit	1	150–300	10–20 Hz packet, 100m range

System	Component	Purpose	Qty	Est. Cost (₹)	Notes
Power Regulation	7805 + capacitors / isolated buck	Clean 5V for ESP32 + sensors	1	200–350	Signal isolation from engine noise critical
Mounting / Integration	3D printed enclosures, shielded cables	Vibration & EMI protection	—	500–1000	Sensor mounts, cable routing, ground plane
Connectors / Wiring	M5/M6 plugs, shielded twisted pair	Reliable sensor interfaces	—	300–500	Waterproof connectors, grounding scheme
V1 Subtotal	—	—	—	**₹3,400–5,500**	Core telemetry stack

2.2 Staged Expansion — V2 and V3

V2 Additions (4–5 weeks in):

- Brake pressure sensor (₹400–800)
- GPS/GNSS module — note: standard GPS is insufficient on tight kart tracks; see Section 8 for RTK discussion
- Secondary wheel sensor (full 4-wheel speed)
- Higher-resolution IMU (ICM-42688 or similar)
- Improved distance sensor (VL53L0X ToF)

V3 — Digital Twin Grade (Week 8+):

- Suspension travel sensors (linear pots)
- Tire temperature sensors (IR, per wheel)

- Tire pressure sensors (TPMS-grade)
- Multi-camera setup (GoPro + onboard stereo)
- Advanced GNSS with RTK capability

2.3 The 10 Signals That Matter Most

If the system had to function with only 10 channels, these are the ones:

1. **Wheel Speed** — vehicle reference; fundamental to lap time and slip
2. **Engine RPM** — power availability, gear point
3. **Steering Angle** — corner characterization
4. **Longitude Acceleration** — braking and traction
5. **Lateral Acceleration** — cornering speed and limit
6. **Yaw Rate** — rotation dynamics
7. **Throttle Position** — driver input for model training
8. **Engine Temperature** — thermal state
9. **Time-to-Contact (TTC)** — obstruction-based driver warning
10. **Track Position** — lap segmentation; *this is an open R&D problem*

2.4 Signal Acquisition & Timestamping Strategy

Critical Rule: Every measurement carries a timestamp from the same reference clock.

- ESP32 internal timer: microsecond resolution
- Sensors sampled at 50–100 Hz (10–20 ms interval)
- Data packed with timestamp before SD write and wireless transmission
- Post-processing: all channels aligned to the same timeline

Why this matters:
Comparing wheel speed at time T against steering angle at the same T requires synchronized clocks, not guessing. Cross-channel desynchronization introduces false correlations in the model.

2.5 Hardware Risks & Mitigations

Risk	Impact	Mitigation
Engine electrical noise → corrupted sensor signals	Kart stops logging mid-run	Shielded wiring, isolated 5V, high-quality caps on ESP32 supply
Vibration → solder joint failures	Intermittent sensor dropout	Vibration-rated mounts, potted connectors, strain relief
Data loss due to SD write failure	Entire run data gone	Checksums on every record; watchdog timer resets ESP32 if stuck
Wireless dropout mid-run	Pit loses live telemetry (but kart keeps logging)	Graceful degradation: pit shows last-known state; full data arrives post-run
Throttle/brake sensor calibration drift	Setup recommendations based on wrong input	Bench-check sensors before every track session

---

3. SOFTWARE & TECH STACK

3.1 Embedded Firmware (ESP32)

Language: C/C++ (Arduino framework or ESP-IDF)
Responsibilities:

- Sensor acquisition (I2C, analog, digital reads)
- Real-time filtering and derived calculations (speed, accel, TTC)
- Timestamp synchronization
- SD card logging (binary or CSV format)
- Wireless packet creation and transmission

- Driver display (OLED HUD updates)
- Fault detection (sensor dropout, low battery, temp threshold)

Architecture:

```
FreeRTOS (dual-core scheduling)
├─ Core 0: Sensor acquisition (interrupt-driven, ~5 ms cycle)
├─ Core 1: Data filtering, derived values, display + wireless
└─ Background: SD write with ring buffer (no blocking I/O)
```

Key Routines:

- `acquire_sensors()` — poll all 9 channels, timestamp each
- `compute_derived()` — speed, accel, TTC calculations
- `filter_data()` — moving average, outlier rejection
- `sd_log()` — binary write to SD (pre-allocated, fast)
- `wireless_send()` — compact packet formation
- `oled_update()` — minimal driver display

Latency Target: Sensor read → calculation → SD write ≤ 20 ms; wireless ≤ 50 ms.

3.2 Pit Dashboard (Python/Web)

Backend: Python + Flask / FastAPI
Purpose: Receive wireless packets, display live gauges, log session metadata

Components:

- Wireless receiver (ESP32 or laptop USB adapter)
- Real-time gauge display (speed, RPM, temperature, TTC)
- Alerts (high temp, TTC warning, sensor fault)
- Session recording (start/stop, metadata)

Technology Stack:

- **Visualization:** Streamlit (rapid prototype) or React (production)
- **Database:** SQLite (sessions, run metadata)
- **Graphing:** Plotly for live traces

3.3 Data & Analytics Pipeline (Python)

Input: SD card data (CSV or binary)
Output: Cleaned dataset, lap-aligned, sector comparisons, model-ready format

Processing Stages:

1. **Ingest** — read raw SD file, validate timestamps
2. **Clean** — remove dropouts, interpolate glitches, outlier filtering
3. **Synchronize** — align all channels to master timestamp
4. **Track Reconstruction** — infer lap position from GPS (if available) or dead reckoning (see Section 8)
5. **Lap Segmentation** — identify start/finish line, break into laps
6. **Sector Analysis** — compare lap sectors, identify performance deltas

   Libraries:
   `pandas`, `numpy`, `scipy`, `matplotlib` / `plotly`

3.4 Digital Twin & Physics Model (Python)

Scope: V1 is a point-mass simulator; V2 can add suspension dynamics.

Model Inputs:

- Track geometry (centerline, curvature, elevation per section)
- Kart parameters (mass, wheelbase, max grip, drag coefficient, engine torque curve)
- Driver behavior (throttle/brake pattern per corner, steering rate)
- Environment (temperature, humidity, surface grip)

Model Outputs:

- Predicted lap time
- Speed through each corner
- G-force envelope
- Throttle and brake zones

Calibration Loop:

1. Measure real lap → extract throttle/braking pattern
2. Simulate lap with guessed tire grip
3. Compare simulated corner speed vs. actual
4. Adjust grip parameter until match within ±5%
5. Use calibrated model to test "what-if" scenarios

Physics Engine: Custom point-mass model (F = ma, friction circle limits, drag equation)

3.5 Recommended Technology Choices

Layer	Technology	Why
Embedded	C++ / ESP-IDF	Real-time, proven on this hardware
Pit Dashboard	Python + Streamlit	Rapid iteration, good enough for student project
Analytics	pandas + numpy	Standard, fast, well-documented
Simulation	scipy + matplotlib	No need for commercial tools (Chrono, Assetto Corsa)
Version Control	Git + GitHub	Mandatory for team coordination

3.6 Data Format & Exchange

On-kart (SD card): Binary format for speed, CSV option for human inspection
Example record (JSON-like, for clarity):

```
{
 "timestamp_us": 12450000,
 "wheel_speed_kmh": 58.3,
 "rpm": 7240,
 "temp_c": 81.5,
 "steering_deg": 13.4,
 "throttle_pct": 82,
 "accel_x_g": 0.72,
 "accel_y_g": 0.91,
 "yaw_rate_dps": 18.4,
 "distance_cm": 142,
 "ttc_s": 2.14,
 "warning_flag": 0
}
```

Wireless (pit): Compact binary (≈ 20 bytes per packet at 20 Hz = ~400 bytes/sec)

---

4. PREDICTIVE COLLISION AWARENESS SYSTEM

4.1 The Innovation Round Feature

This subsystem calculates **time-to-contact (TTC)** in real time and alerts the driver with a steering-wheel-mounted display. It is not an autonomous braking system—it is driver assistance.

4.2 Core Algorithm

```
closing_speed = (previous_distance − current_distance) / time_elapsed
time_to_contact = current_distance / closing_speed   (if closing_speed > 0)
```

Warning Zones (tuned during track testing):

Zone	Condition	Driver Output
SAFE	TTC > 2.5 s, or object moving away	Green display; no alert
CAUTION	TTC 1.0–2.5 s	Yellow steady display + visual indicator
DANGER	TTC < 1.0 s	Red flashing + buzzer

4.3 Hardware Implementation

- **Sensor:** HC-SR04 ultrasonic (V1) or VL53L0X ToF (V2)
- **Display:** 0.96" OLED on steering wheel hub
- **Alert:** Buzzer + LED for redundancy
- **Response Latency:** <50 ms (sense → compute → display)

4.4 Honest Limitations & Testing

- **Does not:** Autonomously control the kart; distinguish between a real threat and a false positive (e.g., trackside cone)
- **Does:** Calculate how fast the gap is closing and warn proportionally
- **Track Testing:** Controlled obstacle approach; measure actual latency; report measured numbers to judges, not estimates
- **Stretch Goal:** Steering angle cross-check (reduce false alarms when driver is clearly steering away)

---

5. FASTF1 INTEGRATION & DATA REFERENCE HARNESS

5.1 What FastF1 Actually Is

FastF1 is a Python library that fetches official Formula 1 telemetry data from FIA public feeds and seasons archives. It is not something we feed into; it is something we learn from.

5.2 How We Use It

1. **Reference Structure:** FastF1 data has the exact same time-aligned structure we need (speed, RPM, throttle, braking, steering) but at higher quality (professional sensors, 1000 Hz)
2. **Prototyping:** We download a real F1 lap, extract throttle/braking patterns, and use them to train our simulator
3. **Validation Benchmark:** We compare our kart model against F1-style lap analysis (sector-by-sector, not absolute speed)
4. **Pipeline Development:** The code we write for FastF1 data teaches us how to structure our own pipeline

5.3 Example Workflow

```
import fastf1
import pandas as pd

# Download a real lap (example: Monaco GP 2023)
session = fastf1.get_session(2023, 'Monaco', 'R')
lap = session.laps.pick_driver('VER').get_car_data().pick_stint(1)

# Extract throttle/brake pattern
throttle_pattern = lap[['Time', 'Throttle', 'Brake', 'Speed']].copy()

# Use this pattern to seed our kart simulator:
# "At this corner, driver applies 82% throttle while doing 150 km/h"
# Our kart: "What speed do we do at this corner if we apply 82% throttle?"
# Compare predicted vs. actual → calibrate grip
```

5.4 Clarity: We Do NOT Feed Our Data Into FastF1

Our kart telemetry is a *separate dataset*. We use FastF1 as a teaching tool and a reference, not as an input source. Our pipeline

is bidirectional (measure → model → predict → validate on real kart), while FastF1 is read-only reference data.

---

6. APIs & DATA INTERFACES

6.1 ESP32 ↔ Pit Wireless Protocol

Packet Structure (compact binary):

```
[Timestamp: 4 bytes] [Speed: 2 bytes] [RPM: 2 bytes] [Temp: 1 byte]
[Steering: 1 byte] [Throttle: 1 byte] [Accel_XY: 2 bytes] [TTC: 2 bytes]
[Flags: 1 byte] = 16 bytes total
```

Frequency: 10–20 Hz (every 50–100 ms)
Range: 50–100 m line-of-sight (nRF24L01+)
Redundancy: If wireless drops, kart continues to log; pit displays last-known state

6.2 SD Card Format

Option A (Fast, Binary):

```
Every record: [timestamp (4B)] [speed (2B)] [rpm (2B)] [temp (1B)] ... [checksum (1B)]
Log file: sequence of records, pre-allocated 1 MB chunks for speed
```

Option B (Human-readable, CSV):

```
timestamp_us,speed_kmh,rpm,temp_c,steering_deg,...
12450000,58.3,7240,81.5,13.4,...
12460000,58.5,7250,81.6,13.2,...
```

Recommendation: Use binary for logging (fast), convert to CSV post-run for analysis.

6.3 Python Import Interface

```
import kurukshetra_kart as kk

# Load a session
session = kk.Session.from_sd_card("session_001.bin")

# Access telemetry
speed = session.telemetry['speed']
rpm = session.telemetry['rpm']

# Align to laps
```

```
laps = session.get_laps()
lap1 = laps[0]
lap1.plot_speed()
```

6.4 Database Schema (SQLite)

```
-- Sessions: metadata for each track outing
CREATE TABLE sessions (
  id INTEGER PRIMARY KEY,
  date DATETIME,
  track_name TEXT,
  driver_name TEXT,
  num_laps INTEGER,
  avg_lap_time REAL,
  telemetry_file TEXT
);

-- Laps: individual lap records
CREATE TABLE laps (
  id INTEGER PRIMARY KEY,
  session_id INTEGER,
  lap_number INTEGER,
  lap_time REAL,
  sector_times TEXT,  -- JSON array
  FOREIGN KEY (session_id) REFERENCES sessions(id)
);
```

---

7. DIGITAL TWIN & CALIBRATION LOOP

7.1 The Four Models

The digital twin combines four sub-models:

Model	Represents	Complexity	Data Source
Track	Centerline, curvature, elevation, surface grip	Geometry from GPS traces or manual mapping	Real laps (wheel speed + steering)

Model	Represents	Complexity	Data Source
Kart	Mass, wheelbase, max lateral G, engine power curve, drag	Spec sheet + calibration	Measured acceleration vs. throttle
Environment	Temperature, humidity, wind, track surface condition	Weather API + manual input	BME280 sensor + pit notes
Driver	Throttle/brake timing, steering rate, corner speeds	Behavior extracted from telemetry	Raw throttle/brake logs

7.2 V1 Simulator — Point-Mass Model

Scope: Not a full vehicle dynamics engine. A simple, buildable model that validates against real telemetry.

Physics:

```
Lateral grip: G_max = μ × 9.81 m/s²  (tire-dependent, tuned via calibration)
Corner speed: V = √(R × G_max × 9.81)   (R = corner radius)
Drag: F_drag = 0.5 × ρ × A × Cd × V²
Acceleration: a = (P_engine / V - F_drag) / mass
```

Simulation Loop:

1. Load track (segments: distance, radius, elevation)
2. For each segment:
   - If straight: accelerate at max power
   - If corner: limit speed to grip circle
   - Integrate distance and time
3. Output: predicted lap time, speed trace, G-force envelope

7.3 Calibration Against Real Data

Goal: Adjust the unknown parameters (tire grip μ, drag Cd) so that *predicted* corner speeds match *measured* corner speeds.

Process:

```
1. Extract one real lap from SD card
2. Identify corners (high steering angle + lateral accel)
3. For each corner:
   - Measured: corner speed (wheel speed + GPS/dead reckoning)
   - Predicted: speed from grip model
   - Error = measured − predicted

4. Adjust grip estimate:
   μ_new = μ_old × (measured_speed / predicted_speed)

5. Re-simulate, compare error
6. Repeat until error < 5% (or accept as "good enough")
```

7.4 Validation Methodology

We will NOT claim:

- Zero latency or instantaneous response
- Guaranteed lap-time improvement
- Perfect prediction in all conditions

We WILL prove:

- Model error < ±5% on corners (validated against measured telemetry)
- Consistent predictions across multiple runs on the same track
- Reproducibility (same conditions → same predicted result)

Test Matrix:

- 2–3 real laps with measured data
- Calibrate model on lap 1
- Predict lap 2 and 3 speeds
- Report measured vs. predicted for each corner
- Calculate RMSE (root-mean-square error)

7.5 What-If Simulation (Scenario Testing)

Once calibrated, the model can answer:

Scenario	Question	Output
Speed sweep	What if we did this corner at 55 km/h instead of 50?	Required G-force, risk of spin
Setup change	Softer suspension = better grip?	Predicted lap time delta

Scenario	Question	Output
Surface condition	What if the track is wet instead of dry?	Grip reduction factor, revised lap time
Driver input	Brake 50 m earlier?	Entry speed change, apex speed change

Important: These are predictions, not guarantees. Always validate on track before trusting a simulator output.

---

8. BOTTLENECKS & CRITICAL R&D ZONES

8.1 Track Position Accuracy — OPEN PROBLEM

The Challenge:
Know, at any moment, where on the track the kart is.

Why It Matters:

- Lap segmentation (identifying start/finish, lap boundaries)
- Sector analysis (which section was slow?)
- Digital twin validation (comparing model predictions to specific track regions)

Current Solutions & Their Limits:

Method	Accuracy	Cost	Limitation
Standard GPS	±5–10 m	₹1500–2000	Way too coarse for a tight kart track
Dead Reckoning (integration of steering + wheel speed)	±2–5 m (drifts over time)	₹200	Cumulative error across a 1 km lap
RTK-GPS (Real-Time Kinematic)	±2–5 cm	₹8000–15000	Very accurate, but expensive for prototyping

Method	Accuracy	Cost	Limitation
LiDAR + SLAM (Simultaneous Localization and Mapping)	±0.5 m	₹20000+	Over-engineered for a student project
Sensor Fusion (Kalman filter combining GPS + IMU + wheel odometry)	±1–2 m	₹2500–4000	Realistic, requires calibration

8.2 Proposed Solution Path (Semester 1)

Phase 1 (Week 2–3): Dead reckoning

- Integrate steering angle and wheel speed
- Accept ±2–5 m error
- Sufficient for rough sector analysis

Phase 2 (Week 5–6): Sensor fusion

- Combine wheel speed + steering + standard GPS (or phone GPS)
- Use Kalman filter to smooth estimates
- Reduces drift, improves accuracy to ±1–2 m

Phase 3 (V2, if time allows): RTK-GPS or manual track mapping

- RTK-GPS: hire equipment for 1–2 track sessions, collect precise baseline
- Manual: walk the track, record waypoints with phone GPS, build reference centerline

Do NOT attempt V1: Full LiDAR SLAM or custom vision-based localization (out of scope).

8.3 Engine Electrical Noise & Signal Integrity

The Challenge:
Ignition system (12V, high-frequency switching) radiates noise that corrupts sensor signals.

Symptoms:

- Temperature sensor reads wildly (80°C, then 150°C, then 80°C)
- RPM count jumps erratically

- Wireless dropouts during acceleration

Solutions (in order of importance):

1. **Shielded cables** — twisted pair, ferrite cores on sensor leads
2. **Isolated power supply** — separate clean 5V for microcontroller and sensors, not shared with ignition
3. **Capacitor bank** — 100µF + 10µF close to ESP32 power pins
4. **Grounding scheme** — single-point ground at battery negative; no multiple ground loops
5. **RF shielding** — aluminum enclosure for ESP32, not open breadboard

Testing: Power up kart engine, monitor sensor readings; acceptable if variation < 2% during idle.

8.4 Wireless Packet Loss & Redundancy

The Challenge:
Radio link drops mid-run (pit loses live telemetry).

Acceptable Outcome:
Pit displays "signal lost" and shows last-known state. Kart continues logging to SD. Full data arrives after run.

Not Acceptable:
Pit thinks kart is stopped when it's still running; or driver HUD goes blank.

Mitigations:

- Watchdog timer on ESP32 (if firmware hangs, auto-reset)
- Redundant alert channel (buzzer on kart, independent of wireless)
- Pre-allocate wireless buffer; allow local operation if transmit fails
- Test range in open field before first track session (ensure 100 m coverage)

8.5 Model Validation Without Crashing

The Challenge:
Simulator may predict behavior that doesn't happen (e.g., "this corner is flat at 80 km/h" when real kart can only do 65).

Solution:
Always test simulator recommendations on the kart before trusting them. Start conservative (slower speeds, softer inputs), measure, then refine.

Process:

1. Simulate: "Try 2° less brake bias"

2. Measure: "What does the kart actually do with that bias?"
3. Compare
4. Next iteration

8.6 Scope Creep Risk

Biggest Risk: Adding V3 features (suspension telemetry, tire temp sensors, advanced ML) before V1 works reliably.

Mitigation: Lock V1 deliverables by Week 2. Treat V2 and V3 as optional. Better to have a rock-solid core system than a feature-rich system that half-works.

---

9. R&D ZONES — Candidate Solutions & Stretch Goals

9.1 Track Reconstruction Algorithm

Candidate A: Dead Reckoning

- Integrate steering angle over time
- Add wheel speed for distance
- Simple, no external dependencies
- Con: Accumulates error (~2–5 m per lap)

Candidate B: Standard GPS + Kalman Filter

- Fuse GPS (±5–10 m) with wheel speed
- Kalman filter reduces noise
- Cost: ~₹1500–2000
- Con: Still ±1–2 m error

Candidate C: Optical Flow (Stretch)

- Small onboard camera, compute flow between frames
- Estimate velocity without relying on GPS
- Con: Requires embedded computer vision (Raspberry Pi?), likely out of scope

9.2 Tire Grip Estimation from Telemetry

Candidate A: Cornering Stiffness Fitting

- Assume linear tire model: Lateral force = Cf × slip angle

- Extract slip angle from steering + yaw rate
- Fit Cf to measured lateral acceleration
- Result: Quantified tire state per lap
- Con: Requires high-fidelity IMU

Candidate B: Machine Learning (V3)

- Train neural net: [Speed, steering, accel, yaw] → predicted grip
- Requires 20–30 laps of diverse data
- Con: Overkill for V1; use simpler physics model

9.3 Weather Compensation

Current: Manual input (pit operator notes "track is wet today")
V2 Option: BME280 temp + humidity sensor onboard
V3 Option: Tire temperature sensing → infer grip from thermal model

R&D Question: Can we reliably predict grip from ambient conditions alone? (Probably not; track surface and rubber buildup matter too much.)

9.4 Advanced Driver Modeling

Simple (V1): Extract throttle/brake input from telemetry; use raw pattern to seed simulator

Advanced (V2): Classify driver behavior per corner type

- Braking zone: aggressive vs. progressive
- Mid-corner: smooth vs. twitchy
- Acceleration zone: early throttle vs. late

Stretch (V3): Predict driver behavior from start of corner (anticipation model)

---

10. ACHIEVABLE IN SEMESTER vs. NOT ACHIEVABLE

10.1 ACHIEVABLE (V1, Weeks 1–6)

- ☑ ESP32 + 9 sensors working reliably
- ☑ SD logging (full dataset, checksummed)
- ☑ Wireless live subset (10–20 Hz) to pit

- ☑ Driver display (steering-wheel OLED)
- ☑ Obstruction detection + TTC calculation
- ☑ Python data pipeline (import, clean, sync)
- ☑ Point-mass simulator (track model + kart model)
- ☑ Calibration loop (adjust grip until model matches real lap ±5%)
- ☑ Pit dashboard (live gauges, basic plotting)
- ☑ Lap segmentation (manual or dead-reckoning based)
- ☑ Sector comparison (lap 1 vs. lap 2 sector times)
- ☑ What-if simulation (speed sweep, surface change)
- ☑ Validation report (measured vs. predicted, error quantified)

10.2 ACHIEVABLE IF WELL-SCOPED (V2, Weeks 7–12)

- ☐ Brake pressure sensor integration
- ☐ Wireless pit dashboard (React frontend, real-time updates)
- ☐ Track reconstruction (GPS + Kalman filter)
- ☐ Session database (storage, query, historical comparison)
- ☐ Predictive ML model (simple neural net for grip)
- ☐ Video sync (telemetry overlay on lap video)

10.3 NOT ACHIEVABLE IN SEMESTER

- ☐ Full vehicle dynamics simulator (suspension, load transfer, tire deflection) — use commercial tool (Project Chrono) if needed
- ☐ Computer vision-based corner detection
- ☐ RTK-GPS baseline (requires external hardware + survey)
- ☐ Autonomous control or braking assist
- ☐ Multi-camera 360° video system
- ☐ Production-grade enclosures (CNC, powder coat)

10.4 The Trade-Off Decision

Option 1: Depth-First (Recommended)
Build V1 rock-solid. Dashboard, telemetry, simulator all trustworthy. Demo end-to-end: measure → model → validate → predict.

Option 2: Breadth-First
Add V2 features quickly. More features to show judges, but less time for validation and testing.

Recommendation: Pick Option 1. A small, bulletproof system beats a large, flaky one.

---

11. SEMESTER ROADMAP & PHASE-GATED TIMELINE

11.1 Overall Cadence

Duration: 3–6 months (14–26 weeks), assuming 3–4 lab days per week per team member
Principle: Each phase must produce measurable, validated output before proceeding to the next.

11.2 Phase-by-Phase Breakdown

PHASE 1: Foundation (Weeks 1–2)

Deliverables:

- ☐ ESP32 + sensor circuit breadboard working
- ☐ Individual sensor validation (wheel speed, RPM, temperature, IMU)
- ☐ First SD card write (≥5 minutes of reliable logging)
- ☐ Real-time serial plot (speed, RPM on terminal)

Team Milestones:

- Embedded team: All sensors reading correctly
- Data team: CSV importer working
- Dashboard team: Pit receiver prototype ready

Go/No-Go Criteria:

- At least 3 sensors logging without dropouts
- Timestamps aligned
- No data corruption over 10-minute test run

PHASE 2: Core Telemetry (Weeks 3–4)

Deliverables:

- ☐ Full 9-channel SD logging, 50–100 Hz
- ☐ Wireless live subset streaming to pit

- ☐ Driver OLED display showing speed, RPM, temp, TTC
- ☐ Pit dashboard displaying live gauges
- ☐ First on-kart integration (wiring, mounts, enclosure)

Team Milestones:

- Embedded team: ESP32 firmware feature-complete for V1
- Integration team: All sensors mounted on actual kart
- Dashboard team: Live pit display working

Go/No-Go Criteria:

- Wireless data arriving at pit with <200 ms latency
- No SD write errors during a 15-minute continuous test
- Driver display readable in sunlight
- At least one successful track test: kart runs, data logged, retrieved

---

PHASE 3: Data Pipeline & Analysis (Weeks 5–6)

Deliverables:

- ☐ Post-run data import (SD card to Python)
- ☐ Automated cleaning (outlier removal, gap filling)
- ☐ Lap segmentation (identify laps automatically or manually)
- ☐ Sector breakdown (time per segment)
- ☐ Comparison plots (lap 1 vs. lap 2: speed trace, G-force)

Team Milestones:

- Python team: Data pipeline tested on real track data
- Dynamics team: Started building physics model

Go/No-Go Criteria:

- Import 10 MB SD file in <5 seconds
- Identify at least 3 complete laps automatically
- Plot speed trace overlaying two laps (visual comparison works)

PHASE 4: Digital Twin V1 (Weeks 7–9)

Deliverables:

- ☐ Point-mass kart model (mass, drag, engine power curve)
- ☐ Track model (centerline, curvature, elevation)
- ☐ Driver model extracted from telemetry (throttle/brake pattern)
- ☐ Basic simulator: predict lap time given these inputs

- [ ] Calibration routine: adjust grip until predicted corner speed matches measured

Team Milestones:

- Dynamics team: Model running, producing speed traces
- Python team: Calibration loop working (error < 5%)

Go/No-Go Criteria:

- Simulate a known corner: predicted speed within ±5% of measured
- Run calibration on a real lap: grip parameter converges
- Explain physical basis (why grip changed from lap 1 to lap 2)

PHASE 5: Validation & What-If (Weeks 10–12)

Deliverables:

- [ ] Model validated against 2–3 different real laps
- [ ] What-if scenarios: speed sweep, setup changes, surface conditions
- [ ] Predict alternative line through one corner; compare to measured
- [ ] Documentation: model assumptions, error sources, confidence limits

Team Milestones:

- Dynamics team: Model trustworthy enough to publish results
- Testing team: Controlled experiments (e.g., "brake 50 m earlier" test)

Go/No-Go Criteria:

- Model RMSE < 5% across 3 test laps (different conditions)
- Predictions consistent (same input → same output)
- Document 3 what-if scenarios with measured validation

PHASE 6: Integration & Polish (Weeks 13–14, if needed)

Deliverables:

- [ ] Full end-to-end demo: run kart → collect telemetry → import data → simulate → predict
- [ ] Pit dashboard enhanced (session history, lap comparison)
- [ ] Technical report and user manual
- [ ] Video demo for judges (on-track footage, simulator playback)

Go/No-Go: System functions reliably from startup to final report generation.

11.3 Milestone Calendar Example

Week	Embedded	Dynamics/Python	Dashboard	Integration
1–2	Sensor bench test	Data import script	Pit receiver code	Thermal design
3–4	Firmware V1 complete	Raw data plots	Live dashboard	Enclosure 3D print
5–6	On-track test 1	Pipeline tested	Session replay	Wiring finalization
7–9	Bug fixes, optimization	Model calibration	Database schema	First wet run
10–12	Telemetry v1.1	Validation suite	Advanced features	Long-duration test
13–14	Release firmware	Final report	Polish UI	Demo prep

11.4 Weekly Team Sync

Every Monday: 30-minute all-hands standup

- Each subteam: what got done, blockers, next week's plan
- Shared decision on scope/priority
- Track against milestone calendar

Every Friday: Integration test

- Run a quick kart session
- Check that all layers work together (sensors → SD → pit → Python)
- Log any bugs, plan fixes

---

12. TEAM ROLES & OWNERSHIP

12.1 Functional Subteams

Subteam	Owner	Responsibilities	Key Skills
Embedded / Electronics	*Name TBD*	ESP32 firmware, sensor interfaces, signal conditioning, power management, wireless protocol	C/C++, embedded systems, circuit design, FreeRTOS
Mechanical / Integration	*Name TBD*	Sensor mounts, enclosures, wiring harness, vibration testing, thermal management	CAD (FreeCAD), 3D printing, mechanical design, EMI/RFI shielding
Telemetry / Networking	*Name TBD*	Pit receiver code, wireless packet format, live dashboard backend, session logging	Python, networking, real-time systems

Subteam	Owner	Responsibilities	Key Skills
Dashboard / UI	*Name TBD*	Frontend visualization, live gauges, historical plotting, user experience	Python (Streamlit or React), UX design
Python / Data Analysis	*Name TBD*	Data import, cleaning, lap segmentation, sector analysis, FastF1 integration	Python (pandas, numpy, scipy), signal processing
Vehicle Dynamics / Simulation	*Name TBD*	Physics model, track geometry, kart parameters, calibration loop, what-if scenarios	Physics, numerical methods, MATLAB/Python, vehicle dynamics
Testing & Validation	*Name TBD*	Controlled track tests, measurement protocol, error quantification, benchmark documentation	Experimental design, data validation, technical writing

12.2 Cross-Functional Dependencies

- **Embedded ↔ Telemetry:** Wireless packet format agreed in Week 1
- **Embedded ↔ Mechanical:** Enclosure & mount designs finalized by Week 2
- **Python ↔ Dynamics:** Data format frozen in Week 3
- **Dashboard ↔ Python:** API defined in Week 4
- **Testing ↔ All:** Track session logistics coordinated every Friday

12.3 Avoiding Bottlenecks

Single point of failure risks:

- One person knows how to upload firmware → cross-train immediately
- Only one person has password to GitHub repo → shared credentials
- One person owns critical design decision (e.g., track position algorithm) → document assumptions

Mitigation:

- Pair programming on critical modules
- Weekly documentation review
- Handoff exercises (person A trains person B on task; then A steps back)

12.4 Communication & Decision-Making

Daily: Slack channel #kart-engineering
Weekly: Monday standup + Friday integration test
Monthly: Formal milestone review (if running longer than 16 weeks)

Decision Process:

1. Problem identified by any subteam
2. Post to #kart-engineering with context
3. Affected teams respond within 24 hours
4. Decision made by team leads at Monday standup
5. Document decision in shared drive (rationale, trade-offs, date)

---

13. COST BREAKDOWN & BUDGET

13.1 V1 Complete BOM (Realistic)

Category	Item	Qty	Unit Cost (₹)	Subtotal (₹)
Controller	ESP32 Dev Board	1	500	500
Sensors	Hall wheel sensor + mag	2	100	200
	Inductive RPM pickup	1	250	250
	DS18B20 temp	1	100	100
	Rotary pot (steering)	1	75	75
	MPU6050 IMU	1	150	150
	HC-SR04 ultrasonic	1	100	100
	Throttle sensor (pot/Hall)	1	150	150
Data & Wireless	MicroSD module	1	150	150
	MicroSD card (32 GB)	1	300	300
	nRF24L01+ module	1	200	200
	External antenna	1	80	80
Display & Alerts	0.96" OLED (I2C)	1	200	200
	Active buzzer	1	30	30
Power & Protection	7805 regulator + caps	1	250	250
	Battery isolation capacitors	1	100	100
Wiring & Mounting	Shielded twisted pair	50 m	20	1000
	Connectors (M5 plugs, etc.)	—	—	400

Category	Item	Qty	Unit Cost (₹)	Subtotal (₹)
	Perfboard, breadboard	—	—	300
	3D printed mounts	—	—	500
Enclosures & Fasteners	Aluminum case (sensor/controller)	1	600	600
	Mounting hardware	—	—	200
	Vibration dampeners	—	—	300
Software & Tools	Git + GitHub (free)	—	0	0
	Arduino IDE / ESP-IDF (free)	—	0	0
	Python libraries (free)	—	0	0
V1 TOTAL				**₹7,935–9,500**

Notes:

- Does NOT include cost of second-tier components (brake pressure sensor, GPS, advanced IMU)
- Does NOT include cost of specialized test equipment (oscilloscope, signal analyzer)
- Does NOT include recurring costs (fuel, track rental, travel)

13.2 V2 & V3 Expansion Costs

V2 Additions (≈₹8,000–12,000):

- Brake pressure sensor: ₹800
- GPS/GNSS module: ₹1500
- Secondary wheel sensors: ₹400
- ICM-42688 IMU (premium): ₹600
- VL53L0X ToF sensor: ₹400
- Development & integration: ₹5000

V3 Additions (₹15,000+):

- Suspension travel sensors: ₹2000
- Tire temperature sensors: ₹3000
- Tire pressure sensors: ₹1500
- RTK-GPS module (rental or purchase): ₹8000–15000
- Advanced ML / Chrono simulator: ₹5000

13.3 Budget Optimization

To stay within ₹15,000–20,000 total:

1. Buy sensors piecemeal (don't bulk-order everything upfront)
2. Use free software (Arduino IDE, Python)
3. 3D print enclosures (don't CNC machine them)
4. Share expensive equipment (oscilloscope) among subteams
5. Defer V3 features until after V1 validation

---

14. REALISTIC vs. ASPIRATIONAL OUTCOMES

14.1 What We Will Definitely Deliver

✅ **Telemetry:** Full 9-channel logging at 50–100 Hz, on-board storage, live wireless subset to pit
✅ **Driver Assistance:** Steering-wheel display with speed, RPM, temperature, TTC warning
✅ **Data Pipeline:** Import, clean, segment, and analyze real track data
✅ **Physics Model:** Calibrated simulator predicting corner speeds within ±5%
✅ **Validation:** Measured error quantified; simulator vs. reality comparison documented
✅ **Integration:** End-to-end system working: run kart → get data → simulate → predict

14.2 What We Might Deliver (If Time & Scope Allow)

⚠ **Advanced Track Position:** Sensor fusion reducing GPS error to ±1–2 m
⚠ **Predictive What-If:** Fast scenario testing (speed sweep, setup change, wet surface)
⚠ **Session Database:** Historical lap comparison, trend analysis
⚠ **Video**

Sync: Telemetry overlay on track video footage
⚠ **Machine Learning:** Neural net predicting tire grip from telemetry

14.3 What We Will NOT Deliver (Explicitly Out of Scope)

❌ **Full Vehicle Dynamics:** Suspension load transfer, tire slip circle, multi-body dynamics
❌ **Autonomous Braking:** The system advises; the driver controls
❌ **Production Enclosures:** We use 3D prints and aluminum cases, not powder-coated housings
❌ **Real-Time GNSS:** We accept ±1–5 m error; no RTK-GPS integration in V1
❌ **Computer Vision:** No camera-based corner detection or vision odometry

14.4 The Honest Pitch to Judges

"We don't just collect telemetry—we convert it into a validated engineering model that predicts kart behavior and guides setup optimization. The closed loop is the innovation: measure → model → predict → validate → improve."

Honesty points:

- We quantify model error (not "it's perfect")
- We measure latency (not "it's instant")
- We identify open problems (track position, tire grip estimation)
- We scope realistically (V1 is small, V2 is ambitious, V3 is dreams)

---

15. TECHNICAL DOCUMENTATION & KNOWLEDGE BASE

15.1 Living Document Standards

Every subteam maintains:

- **DESIGN.md:** Architecture decisions, rationale, trade-offs
- **API.md:** Data formats, interfaces, communication protocols
- **BUILD.md:** Hardware assembly, pin mappings, calibration procedures
- **TEST.md:** Validation procedures, expected results, pass/fail criteria

15.2 GitHub Organization

```
kurukshetra-kart/
├── README.md (overview, getting started)
├── docs/
│   ├── SYSTEM_ARCHITECTURE.md
│   ├── HARDWARE_GUIDE.md
│   ├── SOFTWARE_API.md
│   ├── PHYSICS_MODEL.md
│   └── VALIDATION_REPORT.md
├── firmware/
│   ├── src/
│   │   ├── main.cpp
│   │   ├── sensors.cpp
│   │   ├── telemetry.cpp
│   │   └── display.cpp
│   └── calibration_data.h
├── python/
│   ├── kurukshetra_kart/
│   │   ├── __init__.py
│   │   ├── session.py
│   │   ├── simulator.py
│   │   └── analysis.py
│   ├── tests/
│   │   └── test_pipeline.py
│   └── notebooks/
│       └── example_lap_analysis.ipynb
├── dashboard/
│   └── pit_app.py (Streamlit)
└── hardware/
    ├── schematics/
    ├── pcb_layout/
    └── 3d_models/
```

15.3 Code Review & Quality Standards

- **Python:** PEP 8 style, docstrings on every function, unit tests for data pipeline
- **C++:** Comments on every non-obvious calculation, watchdog timer on firmware, graceful error handling
- **Documentation:** Timestamp every design decision; explain "why," not just "what"

---

16. FINAL RECOMMENDATIONS & SUCCESS CRITERIA

16.1 Success = Closed Loop, Not Feature Count

Judges will care most about:

1. ✅ System actually works: telemetry collected, stored, retrieved without errors
2. ✅ Model validated: predicted behavior matches measured behavior (quantified)
3. ✅ Loop closed: real kart data feeds simulator → simulator predicts → prediction tested on kart
4. ✅ Honesty: errors acknowledged, limitations stated, methodology transparent

Judges will NOT care about:

- ❌ Fancy UI (basic Streamlit is fine)
- ❌ Perfect hardware packaging (3D prints are okay)
- ❌ Every possible sensor (9 core channels is enough)
- ❌ Unrealistic performance claims ("predicts lap time to the millisecond")

16.2 Semester Survival Strategy

Month 1 (Weeks 1–4): Foundation

- Get every sensor working
- First on-kart test
- Establish data pipeline

Month 2 (Weeks 5–8): Core system

- Wireless telemetry reliable
- Physics model calibrated
- Pit dashboard live

Month 3 (Weeks 9–12): Validation & refinement

- Model error < 5%
- What-if scenarios tested
- Documentation complete

If things slip: Cut from V2/V3, polish V1 instead. A tight, validated core beats a rushed, feature-rich mess.

16.3 Demonstration Sequence (For Judges)

1. **Live track run** (if possible): Kart runs; live telemetry arrives at pit dashboard
2. **Data walkthrough:** Import SD card → show raw data, cleaned data, lap segmentation
3. **Model calibration:** "Here's lap 1. We calibrate grip to it. Now here's lap 2. Model predicts corner speeds. Here's the error."
4. **What-if scenario:** "If we did this corner at 5 km/h faster, simulator predicts we need this much more grip. Here's the measurement from a real try."
5. **Honest limitations:** "Track position is ±2 m error. Tire grip estimation is ±10%. These are engineering truths we accept."

16.4 One-Line Pitch

"Real kart. Real data. Real model. Real validation. The closed loop."

APPENDIX A: QUICK REFERENCE — ESP32 PIN MAPPING

```
Pin Assignment (ESP32 Dev Board)

GPIO 35 (ADC) → Wheel speed sensor (Hall voltage)
GPIO 34 (ADC) → Throttle position
GPIO 32 (ADC) → Steering angle potentiometer
GPIO 33 (ADC) → Engine temperature (via OpAmp from DS18B20)

GPIO 26 (PWM) → OLED backlight (optional dimming)
GPIO 25 → Buzzer / Alert tone
GPIO 27 → Status LED

SDA (GPIO 21) → I2C: OLED, MPU6050, DS18B20 adapter
SCL (GPIO 22) → I2C

SPI Bus (SD Card Module):
  MISO (GPIO 19)
  MOSI (GPIO 23)
  CLK (GPIO 18)
  CS (GPIO 5)
```

```
UART (Debug Serial):
  TX (GPIO 1) → USB adapter
  RX (GPIO 3)

nRF24L01+ (SPI-based wireless):
  CE (GPIO 17)
  CSN (GPIO 16)
  (Uses same SPI bus as SD card with separate CS)

RPM Input (Interrupt):
  GPIO 14 → Inductive pickup (pull-up, debounce in firmware)
```

Critical Notes:

- Use GPIO 36 & 39 only for input (no output—they lack pull-up/down)
- Allocate GPIO 4 & 0 as system pins (don't use for sensors)
- Avoid GPIO 6–11 (reserved for flash memory interface)
- SPI bus can multiplex SD card + wireless with separate CS pins

---

APPENDIX B: FASTF1 EXAMPLE — EXTRACTING THROTTLE PATTERN

```
import fastf1
import pandas as pd

# Fetch F1 session (example: 2023 Monaco, Verstappen's qualifying lap)
fastf1.Cache.enable_cache('/tmp/f1_cache')
session = fastf1.get_session(2023, 'Monaco', 'Q')
session.load()

# Get telemetry for one driver
driver = 'VER'
lap = session.laps.pick_driver(driver)[0]
telemetry = lap.get_car_data()

# Extract throttle pattern
throttle_pattern = telemetry[['Distance', 'Speed', 'Throttle', 'Brake']].copy()

# Plot to inspect
import matplotlib.pyplot as plt
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))
```

```
ax1.plot(throttle_pattern['Distance'], throttle_pattern['Speed'])
ax1.set_ylabel('Speed (km/h)')
ax1.set_title(f'{driver} - Speed Trace')
ax1.grid(True)

ax2.plot(throttle_pattern['Distance'], throttle_pattern['Throttle'], label='Throttle')
ax2.plot(throttle_pattern['Distance'], throttle_pattern['Brake'], label='Brake')
ax2.set_xlabel('Distance (m)')
ax2.set_ylabel('Input (%)')
ax2.legend()
ax2.set_title(f'{driver} - Driver Input')
ax2.grid(True)

plt.tight_layout()
plt.savefig('f1_lap_pattern.png', dpi=150)
plt.show()

# Use this pattern to seed kart simulator:
# "At D=500m, F1 car does 150 km/h with 80% throttle"
# Our kart: "At D=500m (similar corner), what speed do we do with 80% throttle?"
# Compare → calibrate kart model
```

APPENDIX C: VALIDATION CHECKLIST

Use this checklist weekly during build and before track testing:

Hardware Validation

- ☐ All 9 sensors reading correctly (serial monitor)
- ☐ Timestamps aligned (all channels stamped at same microsecond)
- ☐ SD write stable for 30 minutes (no corruption, checksum passes)
- ☐ Wireless range ≥100 m line-of-sight
- ☐ OLED display readable in sunlight
- ☐ Buzzer audible at track noise level

Firmware Validation

- ☐ Compile without errors (Arduino IDE or ESP-IDF)
- ☐ Upload to ESP32 without issues

- ☐ Watchdog timer active; resets system if firmware hangs
- ☐ No memory leaks (heap size stable after 1 hour runtime)
- ☐ Graceful degradation (if wireless fails, kart keeps logging)

Data Pipeline Validation

- ☐ Import SD file in <5 seconds
- ☐ Detect and skip corrupted records
- ☐ Interpolate small data gaps (<500 ms)
- ☐ Timestamp continuity check (no jumps backward)
- ☐ Lap detection works (automated or manual)

Model Validation

- ☐ Grip calibration converges (error decreases over iterations)
- ☐ Predicted speed ≥ measured speed in corners (conservative)
- ☐ Model RMSE < 5% on test lap
- ☐ What-if scenario produces sensible results (speed change → time change in right direction)

Integration Validation

- ☐ Run kart for 5 laps; collect full dataset
- ☐ Import data, clean, analyze without crashes
- ☐ Dashboard displays live telemetry correctly
- ☐ Pit and kart clocks synced (time difference < 100 ms)
- ☐ End-to-end demo works: run → collect → import → plot → simulate

---

APPENDIX D: TRACK POSITION DEAD RECKONING ALGORITHM

For V1 (Weeks 5–6), implement simple dead reckoning to estimate lap position:

```
import numpy as np

class DeadReckoner:
    def __init__(self, wheelbase=1.2, track_length=1000.0):
        """
```

```
    wheelbase: distance from rear axle to front axle (meters)
    track_length: approximate perimeter (meters)
    """
    self.wheelbase = wheelbase
    self.track_length = track_length

    self.x = 0.0  # East position
    self.y = 0.0  # North position
    self.theta = 0.0  # Heading (radians)
    self.lap_distance = 0.0
    self.lap_count = 0

def update(self, wheel_speed_ms, steering_angle_rad, dt):
    """
    wheel_speed_ms: speed from wheel sensor (m/s)
    steering_angle_rad: steering angle (radians)
    dt: time step (seconds)

    Returns: (lap_distance, lap_count)
    """
    # Bicycle model: estimate position change
    # tan(steering_angle) gives curvature

    v = wheel_speed_ms
    delta = steering_angle_rad  # steering angle

    # Kinematic bicycle model
    # dx/dt = v * cos(theta)
    # dy/dt = v * sin(theta)
    # dtheta/dt = (v / wheelbase) * tan(delta)

    self.theta += (v / self.wheelbase) * np.tan(delta) * dt
    self.x += v * np.cos(self.theta) * dt
    self.y += v * np.sin(self.theta) * dt

    # Compute distance traveled (Euclidean)
    distance_traveled = v * dt
    self.lap_distance += distance_traveled

    # Detect lap boundary (when distance > track_length, increment lap)
    if self.lap_distance > self.track_length:
        self.lap_distance -= self.track_length
        self.lap_count += 1
```

```
        return self.lap_distance, self.lap_count

# Usage in main telemetry loop:
dr = DeadReckoner(wheelbase=1.2, track_length=1000.0)

for record in telemetry_log:
    speed = record['wheel_speed_kmh'] / 3.6  # convert to m/s
    steering = record['steering_angle_deg'] * np.pi / 180  # to radians
    dt = 0.01  # 100 Hz sampling

    lap_dist, lap_num = dr.update(speed, steering, dt)
    print(f"Lap {lap_num}, Distance {lap_dist:.1f}m")
```

Limitations:

- Error accumulates over time (~±2–5 m per lap)
- Steering angle must be accurate
- Assumes no tire slip (skid invalidates model)

Improvement (V2): Combine with GPS using Kalman filter.

CLOSING STATEMENT

This document is the team's blueprint for building a complete, validated telemetry-to-prediction system in one semester. It prioritizes **integration and validation over feature count**. Success is a working closed loop: measure → model → predict → test → improve.

Key Commitments:

- ✅ Realistic cost (₹18,000–24,000 for V1)
- ✅ ESP32 only (no Arduino Nano)
- ✅ Honest error reporting (model RMSE quantified, not hidden)
- ✅ Semester-scale timeline (3–6 months, phase-gated)
- ✅ Clear team ownership (no overlapping authority)
- ✅ Flagged R&D zones (track position, tire grip) with candidate solutions
- ✅ Achievable V1, ambitious V2, dream V3

Go build something great. 🏎️

Document authored by: Kurukshetra Racing, SKCET
Last updated: September 2026
Revision: 1.0 — Master Build Manual